

CP423 Final Project: Retrieval-Augmented Generation over Material UI Documentation

Cameron Grenier, 169069484

Muhammad Wasay Munawar, 169079709

August 5, 2026

Repository: <https://github.com/CameronGrenier/LLM-Search-Engine> | Demo video:
<https://youtu.be/OGz-0hK1zLA>

1 Introduction and System Overview

A language model answers documentation questions from a memory frozen at its training cutoff, so when a library ships a new major version the model keeps describing the old API and gives no sign that it is doing so. Retrieval-augmented generation fetches passages from an authoritative corpus at query time and constrains the model to answer from them.

Research question: on a documentation corpus that post-dates the generator's training cutoff, does retrieval improve factual accuracy and abstention over a no-context baseline, and how does lexical retrieval compare against dense neural retrieval on identical chunks?

The comparison has three arms: no retrieval, BM25, and dense retrieval. Both indexes are built over the same `chunks.jsonl`, and the generator, its decoding settings, and its prompt scaffolding are held constant, so any difference in output is attributable to retrieval alone. The pipeline runs in two phases, indexing once over the corpus and query serving per question, and is drawn in Appendix A.

2 Corpus Construction

2.1 Source and preprocessing

The corpus is the Material UI documentation at tag v9.2.0: 151 markdown prose files and 143 API reference JSON files, 294 documents in total. `fetch_corpus.py` downloads one codeload tarball rather than walking the GitHub API file by file, which would need several hundred requests and hits unauthenticated rate limits. The tarball is cached and the tag is pinned in `config.py`, so later upstream edits cannot change the experiment.

MUI was chosen for version drift and verifiability. It ships breaking changes quickly and v9.2.0 post-dates the generator's cutoff, which creates headroom between what the model knows and what the docs say. Every answer we ask for is a prop name, a default value, or a version number checkable against a line of a pinned file, so grading is not a judgment call.

`preprocess.py` treats the two file types differently. Prose is MDX, so embedded HTML is stripped with BeautifulSoup before anything else runs, since a tag-stripping regex cannot tell an HTML wrapper from a JSX element. Frontmatter supplies title and component metadata. API JSON is flattened into sentences of the form `Prop 'max': type number. Default: 99`, because the retrievers would otherwise match JSON punctuation rather than field meaning, and these files carry most of the factoid answers.

Demo directives get particular handling. MUI embeds interactive examples with a directive naming a source file, and the obvious move is to inline that source at index time. We tried it and rejected it: every demo opens with a near-identical import and export block, so hundreds of chunks came to share a large common component in embedding space, which compresses the distances retrieval depends on, and the added code pushed documents past the embedder's token limit. Each directive is instead replaced by a position marker and the source is stored separately in `demos.jsonl`, so the index sees prose only while the marker records where the code sat. The code is spliced back into the retrieved passage at generation time, since a question about how to do something in MUI is answered by the example rather than by the prose describing it. Retained per document are title, component, repository path, document type, and a deterministic identifier derived from the filename, which is what makes a citation traceable back to a pinned file.

2.2 Chunking policy

Prose is split on headings. Each chunk holds two consecutive heading sections and adjacent chunks overlap by one heading, so no boundary separates a claim from the material framing it. Sections longer than 500 tokens are divided further with a 100-token overlap. API records are kept whole, since a prop name loses meaning once separated from its type, default, and description. This produces 2140 chunks, 1997 prose and 143 API, averaging 104 tokens. Both retrievers index this same set, so chunking is held constant across the comparison. Counts are in Appendix C.

3 Corpus Suitability Diagnostic

RAG is only worth measuring if the corpus holds information the generator does not already have. The ten answerable evaluation questions, 7 factoid and 3 multi-hop, were put to Qwen2.5-3B-Instruct with no retrieved context and the naive system prompt.

None of the ten was fully correct. The model declined three and answered seven wrongly, volunteering plausible but invented prop names and defaults such as `maxCount` for `Badge`, `count` for `AvatarGroup`, and `step` for both `Rating` and `Slider`. Adding retrieval improved 9 of the 10: dense RAG answered 8 fully correctly and BM25 answered 4. The corpus is therefore not memorised at v9.2.0, and the gains measured in Section 7 are attributable to retrieval rather than to parametric knowledge. Counts are in Appendix D.

4 Retrieval

4.1 BM25

BM25 runs over the same 2140 chunks with $k_1 = 1.5$ and $b = 0.75$. Tokenisation is a lowercased word-boundary split and nothing more. There is no stopword list and no lemmatiser, because both retrievers share one chunk text and aggressive normalisation would help the lexical side while damaging the dense side, which needs natural word order.

The two parameters control the two ways BM25 departs from raw TF-IDF. Term frequency saturates under k_1 , so the tenth occurrence of a term adds almost nothing over the ninth and a chunk cannot win by repetition alone. Length normalisation under b divides by chunk length relative to the corpus average, so a long chunk does not accumulate score merely by containing more words. Both matter here, where API chunks are long and repeat prop names while prose chunks are short.

The index holds a term-frequency counter per chunk plus a corpus-wide document-frequency table, and a query is scored by a linear scan over every chunk. That is naive next to a postings-list lookup, but at this size it answers in milliseconds and needs no model, no GPU, and no re-indexing when the generator changes. BM25 is strongest on exact component and prop names, where a rare token carries high inverse document frequency, and weakest when a question is worded differently from the documentation it needs to find.

4.2 Dense retrieval

Dense retrieval embeds chunks and queries into one space and treats relevance as proximity, matching paraphrase where BM25 matches tokens. The model is `jinaai/jina-embeddings-v3-hf`, 1024 dimensions with an 8192-token limit. Retrieval is asymmetric: corpus text uses the `retrieval_passage` LoRA adapter and queries use `retrieval_query`, so question-shaped text lands near answer-shaped text. Only one adapter may be loaded per process, since loading a second re-initialises the first and LoRA zeroes its B matrix, silently leaving the two sides in different spaces, so the adapter is selected once through an environment variable set before the module is imported.

Chunk vectors come from late chunking. MUI docs contain many near-identical passages, and a chunk embedded in isolation yields a vague vector because its tokens attend only to the chunk; late chunking pools each vector from a forward pass over the surrounding document instead. The four documents that exceed the token limit, the largest at 26,535 tokens, are covered by windows: whole chunks are packed backwards from the document end until coverage would exceed 6000 tokens, so every chunk is owned by exactly one window, and each window start is then extended backward toward an 8000-token ceiling as read-only context. Both operations run backward because documentation is read top to bottom.

Vectors go into a FAISS `IndexFlatIP`, exact search, since an approximate index would trade recall for speed the system does not need at 2140 vectors, and L2 normalisation makes inner product equal cosine similarity (Appendix G). The payoff is paraphrase matching: dense retrieval put the `AvatarGroup` `max` chunk first for a question about limiting how many avatars are shown, where BM25 ranked it sixty-first, while BM25 wins on rare exact tokens such as `enterTouchDelay`. The costs are 201 seconds of indexing, a full re-embed after any chunking change, and the 8192-token ceiling that Section 9 returns to.

5 Generation

Generation uses Qwen/Qwen2.5-3B-Instruct, named in `config.py` and loaded locally through Hugging Face `transformers` with `dtype auto`, on CUDA, Apple MPS, or CPU in that order; our runs were on MPS. It was chosen as an ungated checkpoint small enough to run unquantised on a laptop and instruction-tuned enough to obey a prompt confining it to the supplied passages. Decoding is greedy, `do_sample=False` with

`max_new_tokens=512`, a reproducibility requirement rather than a quality choice, since sampling would make the manual judgments in Section 7 unrepeatable. `src/seeding.py` applies seed 42 to the Python, NumPy, and Torch generators when either model loads, and `reproduce.py` sets `PYTHONHASHSEED` per stage; no stage is actually stochastic, so the seeding is insurance.

`TOP_K` is 5. Passages are numbered `[n]` (`chunk_id`) and carry the chunk text followed by any demo source that chunk referenced, raising the mean prompt from roughly 900 tokens to 2400 against a 32,768-token window. Model, device, decoding settings, seed, passage formatting, and system prompt are identical across the BM25 and dense conditions, and only the retrieved set differs. The naive condition uses its own prompt, since the RAG prompt would otherwise make it abstain for reasons unrelated to its knowledge. Both are in Appendix B.

One decision deserves stating plainly, because it makes our numbers look worse. An earlier version replaced the answer with `I don't know` whenever it carried no valid citation or hedged, and ran a second pass in which the same model verified its own answer. Both were removed: the rewriting made the inline citation rate true by construction rather than measured, and the verifier approved a reply consisting of nothing but the string `[1]`, `[2]` as supported by the passages it cited. The refusal instruction now lives in the prompt, the model's reply is recorded verbatim, and grounding is judged by a human.

6 Evaluation Set

The gold set is 13 questions written by hand while reading the corpus: 7 factoid, 3 multi-hop, and 3 unanswerable. Each carries a reference answer, a type, and the ground-truth chunk ids that support it. Multi-hop questions require evidence from two chunks, so a system must retrieve both to answer. Unanswerable questions have no valid supporting chunk anywhere in the corpus and take `I don't know` as the reference answer, testing abstention rather than recall. The full set is in Appendix E.

Several sources of bias should be read alongside the results. The set is small, so one question shifts any metric by roughly eight percentage points. Component coverage is uneven and reflects what was interesting to read rather than a sample of the library. The questions were written and graded by the same people who built the system, and scoring is manual and therefore subjective. Exact gold matching also penalises a retriever that returns a neighbouring chunk carrying comparable evidence, which our one-heading overlap makes common.

7 Results

Retrieval was scored on the ten answerable questions. Dense retrieval leads on recall at 0.700 against 0.650 and on precision at 0.180 against 0.160, but the two are effectively tied on `nDCG@5` at 0.581 against 0.584, so dense finds slightly more of the required evidence while ranking it no better. The margin is narrower than the design predicted, and it rests on single questions: dense misses the Button version chunk and the Tooltip chunk that BM25 finds, and finds the AvatarGroup and Badge API chunks that BM25 buries.

Judged against the reference answers, dense RAG is fully correct on 10 of 13 questions, BM25 on 7, and the no-context baseline on none. The automated metrics split differently. BM25 leads on exact match at 0.231 against 0.154 and on mean token F1 at 0.665 against 0.504, because its failures are terse and hedged while dense answers at greater length, which token overlap punishes even when the answer is right. Full tables are in Appendix F.

The unanswerable questions separated the two systems. BM25 declined all three with the exact fallback wording. Dense declined two and, on the Button variants question, asserted that the fourth variant is the Floating Action Button and attached a citation to it, which is exactly the fabrication the question was written to catch. The baseline declined none.

The clearest result concerns citations, and it is not a good one. Only 2 of 9 non-refusal BM25 answers and 1 of 10 dense answers carry any inline citation marker at all, and almost none of those markers land on a gold chunk, giving citation precision of 0.111 and 0.000. The model overwhelmingly writes correct prose and simply omits the citation the prompt asks for. Our earlier pipeline reported a 100 percent citation rate, but that figure was produced by rewriting every uncited answer into a refusal before it was recorded. Removing that rewrite replaced a manufactured number with a measured one, and the measured one is poor.

8 Error Analysis

Failures separate into three kinds with different causes. Retrieval failures mean the evidence never reached the model, generation failures mean it reached the model and was not used, and citation failures mean the answer was right but not traceable to the passage supporting it.

Refusing with the answer in hand. On Q1 the gold chunk ranked first under both retrievers and contains the install command verbatim, and both systems replied `I don't know`. The passage states that Material UI ships no menu bar and that Base UI must be installed, so the evidence was present and legible. The refusal instruction is the likely cause: it asks the model to decline unless the passages explicitly answer the question, and a two-part question appears to trip that test whenever either part looks underspecified. This is the cost of moving refusal into the prompt, and it is a prompt problem rather than a retrieval one.

Fabricating a value the context does not contain. On Q7 BM25 missed the AvatarGroup API chunk, which is the only place the default of 5 appears, and the model answered that the default is 6 with a citation attached. On Q6 the same retriever returned prose chunks that name `max` without its default, and the model reported that no default is specified. The pattern is that a missing second fact produces either an invention or a hedge, never a refusal. Q13 is the same failure on an unanswerable question: dense retrieval returned Button prose, and the model named the Floating Action Button as a fourth Button variant and cited a passage for it.

Multi-hop retrieval. All three multi-hop questions still lose a hop under at least one retriever. BM25 recovered one of two chunks on all three, ranking the Rating and Slider API chunks twenty-third and one hundred and eleventh, and in both cases the model named the right prop and could not give the default. Dense recovered both Rating chunks and answered correctly, but lost a hop on Slider and Masonry. The cause is structural, since both retrievers rank a flat list against a single query representation, so a two-hop question is dominated by its first hop and the second competes against chunks matching the first. Raising k surfaces the missing evidence but dilutes the prompt.

Citation faithfulness. This is now the system's weakest dimension. Across nineteen non-refusal RAG answers only three carry an inline citation marker, and only one of those lands on a gold chunk. The model produces the prose the prompt asks for and silently drops the citation requirement, most often when the answer is a single copied line such as `Prop 'max': type number. Default: 99`. Because nothing rewrites the answer any more, an uncited answer now reaches the results file as an uncited answer, which is what makes the rate measurable at all. Separately, every citation record previously carried a null `doc_id`, because the citation builder read the field defensively while neither retriever emitted it. Both retrievers now return `doc_id` and the field is read directly, so the same mismatch would fail immediately rather than write nulls into every record.

9 Limitations and Future Work

The binding constraint on the dense side is jina-v3's 8192-token window. Four documents exceed it, the largest at 26,535 tokens, and they require the backward-packing windows of Section 4.2, which complicate indexing and leave those chunks without whole-document context. A larger-context embedder such as jina-embeddings-v4 or Qwen3-Embedding-8B, both around 32K tokens, would let every document in this corpus embed in one pass. The cost is real: both are substantially larger models, so memory and indexing time rise, adopting one requires a full re-embed, and a 32K window moves the wall rather than removing it.

The multi-hop failures point at concrete work rather than a redesign. Hybrid retrieval fusing the BM25 and dense rankings would recover evidence that one retriever ranks well and the other does not, which fits the Masonry case where BM25 ranked the missing chunk thirteen places higher. Reranking a larger candidate pool with a cross-encoder would let the system look past the top five without paying the prompt-dilution cost. Retrieving neighbouring chunks alongside each hit, or decomposing a multi-hop question into subqueries, attacks the same problem from the query side.

Citation compliance is the other clear target. A 3B model asked to answer, refuse, and cite in one pass reliably does the first two and drops the third, so the practical fix is to stop asking it to do all three at once: generate the answer, then run a separate pass whose only job is to attach passage numbers to sentences already written.

The rest are limits of scope: a single technical domain, a 13-question evaluation set, a fixed top-five cutoff, manual and subjective scoring, and the capacity of a 3-billion-parameter generator. One reproducibility caveat belongs here too. Dense retrieval results shifted slightly between runs with no change to the retrieval code, moving `recall@5` from 0.800 to 0.700, which we attribute to accelerator-level floating-point variation in the embedding forward passes rather than to anything the seed controls.

10 Conclusion

Retrieval substantially improved answer quality on this corpus. Dense RAG was fully correct on 10 of 13 questions against 7 for BM25 and none without context, and the diagnostic showed the same effect from the other direction, with retrieval fixing 9 of the 10 answerable questions the generator could not answer from memory.

The result is not that dense retrieval dominates. Its retrieval advantage is thin, a recall edge of 0.700 against

0.650 with nDCG effectively tied, and BM25 remains cheaper by every practical measure, leads on both automated generation metrics, and declined all three unanswerable questions where dense fabricated a Button variant. Both systems depended on retrieving every required chunk within the top five, which is where multi-hop questions broke. The honest headline is that grounded answers are common and grounded *citations* are rare: only three of nineteen substantive RAG answers cite anything at all, which is the first thing we would fix.

A System Pipeline Diagram

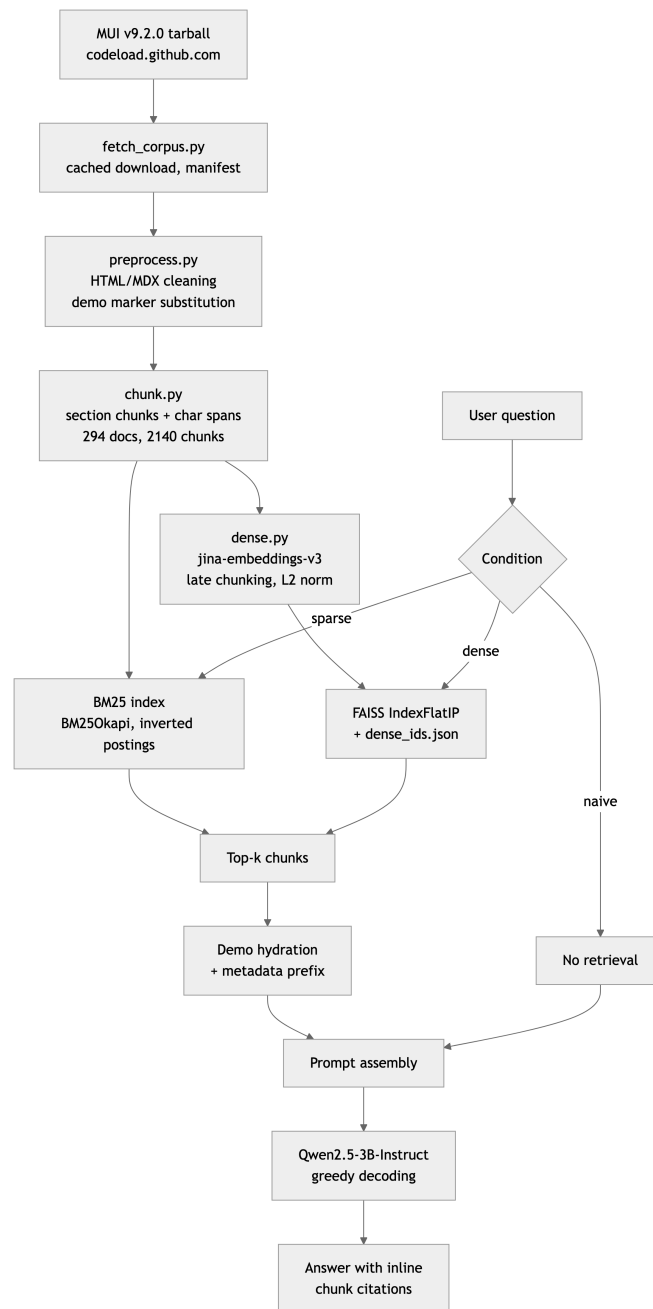


Figure 1: System pipeline. Indexing runs once over the corpus. Query serving branches on retrieval condition and converges on a single generator. Authored in Mermaid; source in `pipeline.mmd`.

B Prompts

System prompt, RAG conditions:

```
Answer the question using only the supplied numbered passages.

If the passages do not explicitly answer the question, your entire reply must be
this exact sentence:

I don't know

Write it exactly that way: a straight apostrophe, no trailing period, no citation,
no explanation, and nothing before or after it.

Otherwise, answer the question and follow these rules:
- Answer every part of the question that is explicitly supported.
- Place an inline citation such as [1] immediately after every factual sentence.
- Use only passage numbers that were supplied.
- Do not use prior knowledge.
- Do not infer prop names, defaults, variants, or component behaviour from related components.
- Do not treat a passing mention, dependency list, or example name as an answer.
- Never reply with citation markers alone; every reply must contain a sentence.
- Keep the answer concise.
```

System prompt, naive condition:

```
Answer the question about Material UI as accurately as you can.
Keep it concise and to the point.
If you do not know the answer, respond with "I dont know"
```

User message, RAG conditions. Each passage is the chunk text prefixed by its number and chunk id, followed by any demo source that chunk referenced:

```
Numbered passages:
[1] (prose__accordion__chunk_0004)
<chunk text>

```jsx
<demo source referenced by this chunk>
```

[2] (api__tooltip__chunk_0001)
<chunk text>
...

Question:
<question text>

Answer:
```

C Corpus and Chunk Statistics

| Source type | Count | Chunk type | Count |
|-----------------------|-------|------------|-------|
| Markdown files | 151 | Prose | 1,997 |
| API JSON files | 143 | API | 143 |
| Referenced demo files | 1,567 | Total | 2,140 |
| Searchable documents | 294 | | |

Table 1: Corpus sources and resulting chunk counts. Average chunk length is 104 whitespace tokens; 613 of the extracted demo files are referenced by a directive and stored in `demos.jsonl`.

| No-context result | Count | System | Fully correct |
|--------------------|-------|------------|---------------|
| Fully correct | 0 | No context | 0/10 |
| Declined to answer | 3 | BM25 RAG | 4/10 |
| Incorrect | 7 | Dense RAG | 8/10 |

Table 2: Diagnostic on the ten answerable questions, 7 factoid and 3 multi-hop. Left: breakdown with no retrieved context. Right: fully correct answers by condition.

D Corpus Suitability Diagnostic Results

E Evaluation Set

The 13 hand-written questions with their gold answers and the chunks that support them. Multi-hop questions require evidence from two chunks. Unanswerable questions have no supporting chunk anywhere in the corpus.

Factoid

Q1 (*prose__menubar*)

Does Material UI include a Menubar component out of the box, and what do I need to install to use the menubar shown in the docs?

Gold answer: No. Material UI does not ship a menu bar component. The docs page composes the Base UI Menubar styled to Material Design, so Base UI must be installed first with `npm install @base-ui/react`.

Q2 (*prose__progress*)

Which prop keeps the CircularProgress track visible at all times?

Gold answer: The `enableTrackSlot` prop.

Q3 (*prose__buttons*)

From which Material UI version is the loading prop available on Button?

Gold answer: Starting from v6.4.0.

Q4 (*prose__tooltips*)

Why does showing a tooltip on touch devices require a long press?

Gold answer: Because `enterTouchDelay` defaults to 700 ms.

Q5 (*prose__avatars*)

How do I customize the surplus avatar in an AvatarGroup, and what argument does the callback receive?

Gold answer: Pass a callback to `renderSurplus`. It receives the surplus number, derived from the children and the `max` prop, and returns a `React.ReactNode`.

Q6 (*api__badge*)

Which Badge prop caps large numeric values, and what value does it cap at by default?

Gold answer: The `max` prop, which defaults to 99.

Q7 (*api__avatar-group*)

Which prop limits how many avatars an AvatarGroup shows, and what is its default value?

Gold answer: The `max` prop, which defaults to 5.

Multi-hop

Q8 (*prose__rating, api__rating*)

Which Rating prop sets the minimum increment the value can change by, and what is that prop's default?

Gold answer: The `precision` prop, which defaults to 1.

Q9 (*prose__slider, api__slider*)

Which Slider prop controls stepping with Page Up/Down or Shift plus arrow keys, and what is its default compared to the default step?

Gold answer: The `shiftStep` prop, which defaults to 10 while `step` defaults to 1, so the default `shiftStep` is ten times larger.

Q10 (*prose__masonry, prose__image-list*)

Masonry orders its children by row. Which component should I use instead to order images by column, and what determines the container heights there?

Gold answer: Use `ImageList`, specifically the masonry image list variant, where container heights are sized dynamically to reflect each image's aspect ratio.

Unanswerable

Q11 (*none*)

What does the type prop do in the Card MUI component?

Gold answer: I don't know.

Q12 (*none*)

How do I use the TreeView MUI component?

Gold answer: I don't know.

Q13 (*none*)

The MUI Button component comes with four variants: text, contained, outlined, and one other. What is this fourth variant?

Gold answer: I don't know.

F Full Results Tables

| Retriever | Precision@5 | Recall@5 | nDCG@5 |
|-----------|-------------|----------|--------|
| BM25 | 0.1600 | 0.6500 | 0.5839 |
| Dense | 0.1800 | 0.7000 | 0.5807 |

Table 3: Retrieval metrics on the ten answerable questions. The three unanswerable questions are excluded, since they have no gold chunk.

| Metric | No context | BM25 RAG | Dense RAG |
|------------------------------------|------------|----------|-----------|
| Fully correct | 0/13 | 7/13 | 10/13 |
| Exact match | 0.0000 | 0.2308 | 0.1538 |
| Mean token F1 | 0.3450 | 0.6650 | 0.5037 |
| Unanswerable refusal accuracy | 0.000 | 1.000 | 0.667 |
| Inline citation rate, non-refusals | n/a | 0.222 | 0.100 |
| Mean citation precision | n/a | 0.111 | 0.000 |
| Mean citation recall | n/a | 0.050 | 0.000 |

Table 4: Generation results across all 13 questions per condition. Fully correct is judged against the reference answer; the remaining rows are computed automatically. The baseline scores 0.000 on refusal accuracy because it writes `I dont know` without the apostrophe, which the exact-match test rejects.

G Dense Index Statistics

| Metric | Value |
|--|----------------------------|
| Documents | 294 |
| Chunks | 2140 |
| Documents embedded in a single pass | 290 |
| Documents requiring windows | 4 |
| Forward passes over windows | 305 |
| Chunks pooled from a window | 2140 |
| Chunks embedded independently (fallback) | 0 |
| Embedding dimension | 1024 |
| Longest document | 26,535 tokens |
| Indexing time | 201 s (Apple Silicon, MPS) |

Table 5: Dense index statistics from the final clean run.